

Program Structures and Algorithms
Spring 2025

NAME: Xinduo Fan

NUID: 002059620

GITHUB LINK: <https://github.com/XDagain-jpg/DSAIPG>

Assignment 1 (Random Walk) Task:

1. Implement three (3) methods (*repeat*, *getClock*, and *toMillisecs*) of a class called *Timer*. Pass the related unit tests.
2. Implement insertion sort (in the *InsertionSortComparator* class) with the *helper.swapStableConditional* method, continuing if it returns true, otherwise breaking the loop. Pass the unit tests in *InsertionSortTest*.
3. Implement a main program (in our case, *InsertionSortBenchmark_Assignment3.java*) to run the following benchmarks: measure the running times of this sort, using four different initial *Integer* array ordering situations: random, ordered, partially-ordered and reverse-ordered. Use the doubling method for choosing *n* and test for at least five values of *n* (in our case, $n = \{500, 1000, 2000, 4000, 8000\}$). Draw any conclusions from your observations regarding the order of growth, see later conclusion section for more details.

Code Screenshots:

Part 1) For Timer.java (3 methods):

```
public <T, U> double repeat(int n, boolean warmup, Supplier<T> supplier, Function<T, U>
//IMPLEMENTED : note that the timer is running when this method is called and should

    pause();

    if (warmup) {
        for (int i = 0; i < n; i++) {
            T input = supplier.get();
            if (preFunction != null) {
                input = preFunction.apply(input);
            }
            function.apply(input);
        }
        final double result = meanLapTime();
        resume();
        return result;
    }

    for (int i = 0; i < n; i++) {
        T input = supplier.get();
        if (preFunction != null) {
            input = preFunction.apply(input);
        }

        resume();
        U output = function.apply(input);
        pauseAndLap();

        if (postFunction != null) {
            postFunction.accept(output);
        }
    }

    final double result = meanLapTime();
    resume();
    return result;
}

private static long getClock() {
    return (long) System.nanoTime();
    // END SOLUTION
}

private static double toMillisecs(long ticks) {
    return (double) ticks / 1_000_000.0;
    // END SOLUTION
}
```

Part 2) For InsertionSortComparator.java (1 method) :

```
public void sort(X[] xs, int from, int to) {

    final Helper<X> helper = getHelper();

    //Comparator<X> comparator = helper.getComparator();

    for (int i = from + 1; i < to; i++) {
        for (int j = i; j > from; j--) {
            boolean swap = helper.swapStableConditional(xs, j);
            if (!swap) break;
        }
    }
}
```

Part 3) For benchmarking insertion sort (InsertionSortBenchmark_Assignment3.java):

//first 2 pics are where the benchmarking happens,

```
public static void main(String[] args) throws IOException {

    int[] sizes = {500, 1000, 2000, 4000, 8000};

    Config config = Config_Benchmark.setupConfigFixes();

    System.out.println("n,ordering,time(ms)");
    for (int n : sizes) {
        benchmark(n, config);
    }
    System.out.println("Done.");
}

private static void benchmark(int n, Config config) {

    runBenchmark("random", n, () -> generateRandomArray(n));
    runBenchmark("ordered", n, () -> generateOrderedArray(n));
    runBenchmark("partially-ordered", n, () -> generatePartiallyOrderedArray(n));
    runBenchmark("reverse-ordered", n, () -> generateReverseOrderedArray(n));
}

private static void runBenchmark(String orderingType, int n, Supplier<Integer[]> supplier) {

    Benchmark_Timer<Integer[]> benchmark = new Benchmark_Timer<>(
        "Insertion Sort " + orderingType,
        array -> {
            Integer[] copy = Arrays.copyOf(array, array.length);
            new InsertionSort<Integer>().mutatingSort(copy);
        }
    );

    Integer[] array = supplier.get();

    double timeMs = benchmark.runFromSupplier(() -> Arrays.copyOf(array, array.length), 10);

    System.out.printf("%d,%s,%.3f%n", n, orderingType, timeMs);
}
```

Note: here we use the runFromSupplier method, which includes a warm up section with n/15 (which gives us warm up runs more than 10 times as required in the instruction).

//second 2 pics are for generating the required array

```
/**
 * Generates a randomly generated array of size n.
 */
private static Integer[] generateRandomArray(int n) {
    Random random = new Random();
    Integer[] array = new Integer[n];
    for (int i = 0; i < n; i++) {
        array[i] = random.nextInt(10000);
    }
    return array;
}

/**
 * Generates an ordered (ascending) array of size n.
 */
private static Integer[] generateOrderedArray(int n) {
    Integer[] array = new Integer[n];
    for (int i = 0; i < n; i++) {
        array[i] = i;
    }
    return array;
}
```

```

/**
 * Generates a partially ordered array, to simplify this process,
 * we'll let the first half of array sorted, the second half random.
 */
private static Integer[] generatePartiallyOrderedArray(int n) {
    Integer[] array = new Integer[n];
    Random random = new Random();

    for (int i = 0; i < n / 2; i++) {
        array[i] = i;
    }

    for (int i = n / 2; i < n; i++) {
        array[i] = random.nextInt(10000);
    }
    return array;
}

/**
 * Generates a reverse-ordered (descending) array.
 */
private static Integer[] generateReverseOrderedArray(int n) {
    Integer[] array = new Integer[n];
    for (int i = 0; i < n; i++) {
        array[i] = n - i;
    }
    return array;
}

```

Unit Test Screenshots:

For part 1):

Finished after 2.19 seconds

Runs: 11/11 Errors: 0 Failures: 0

com.phasmidsoftware.dsaipg.util.TimerTest [Runner: JUnit 4] (2.153 s)

- testPauseAndLapResume0 (0.183 s)
- testPauseAndLapResume1 (0.315 s)
- testLap (0.210 s)
- testPause (0.211 s)
- testStop (0.106 s)
- testMillisecs (0.102 s)
- testRepeat1 (0.123 s)
- testRepeat2 (0.240 s)
- testRepeat3 (0.195 s)
- testRepeat4 (0.362 s)
- testPauseAndLap (0.106 s)

Finished after 0.642 seconds

Runs: 7/7 Errors: 0 Failures: 0

com.phasmidsoftware.dsaipg.util.BenchmarkTest [Runner: JUnit 4] (0.628 s)

- testRunFromSupplier_ZeroIterations (0.074 s)
- testRunFromSupplier_FullPipeline (0.001 s)
- testWaitPeriods (0.550 s)
- testRunFromSupplier_WithPostprocessing (0.002 s)
- getWarmupRuns (0.000 s)
- testRunFromSupplier_WithPreprocessing (0.001 s)
- testRunFromSupplier_Basic (0.000 s)

For part 2):

Finished after 0.105 seconds

Runs: 8/8  Errors: 0  Failures: 0

✓  com.phasmidsoftware.dsaipg.sort.elementary.InsertionSortComparatorTest [Runner: JUnit 4] (0.092 s)

- ✓  testSortReversedArray (0.090 s)
- ✓  testSortEmptyArray (0.000 s)
- ✓  testSortWithSubArray (0.000 s)
- ✓  testSortWithDuplicateValues (0.000 s)
- ✓  testSortWithStrings (0.001 s)
- ✓  testSortAlreadySortedArray (0.000 s)
- ✓  testSortSingleElementArray (0.000 s)
- ✓  testSortWithIntegers (0.001 s)

Finished after 0.112 seconds

Runs: 11/11  Errors: 0  Failures: 0

✓  com.phasmidsoftware.dsaipg.sort.elementary.InsertionSortTest [Runner: JUnit 4] (0.099 s)

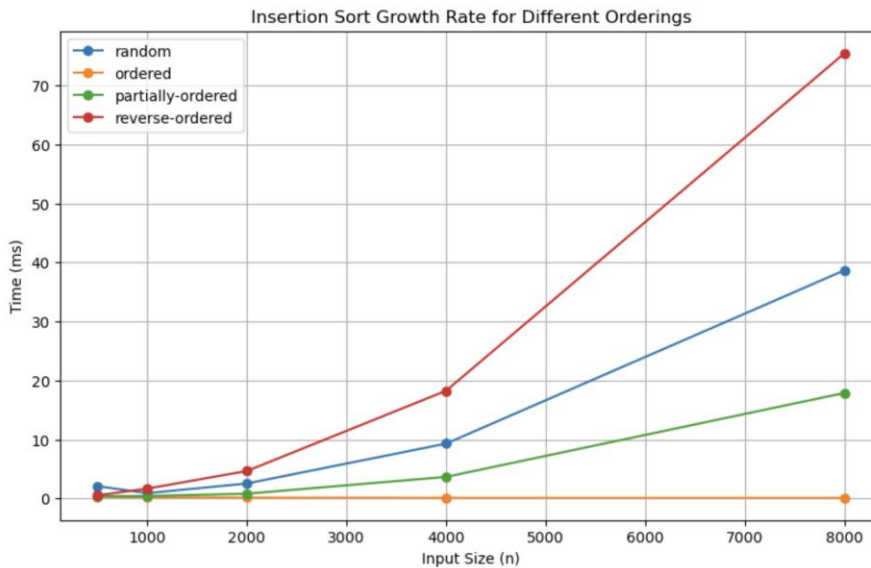
- ✓  testMutatingInsertionSort (0.083 s)
- ✓  sort0a (0.003 s)
- ✓  sort0b (0.001 s)
- ✓  sort0c (0.000 s)
- ✓  sort4a (0.000 s)
- ✓  sort4b (0.001 s)
- ✓  sort0 (0.000 s)
- ✓  sort1 (0.004 s)
- ✓  sort2 (0.004 s)
- ✓  sort3 (0.001 s)
- ✓  testStaticInsertionSort (0.001 s)

Observations:

1	n,ordering,time(ms)
2	500,random,2.096
3	500,ordered,0.308
4	500,partially-ordered,0.378
5	500,reverse-ordered,0.545
6	1000,random,0.909
7	1000,ordered,0.216
8	1000,partially-ordered,0.431
9	1000,reverse-ordered,1.680
0	2000,random,2.546
1	2000,ordered,0.158
2	2000,partially-ordered,0.819
3	2000,reverse-ordered,4.665
4	4000,random,9.302
5	4000,ordered,0.116
6	4000,partially-ordered,3.666
7	4000,reverse-ordered,18.258
8	8000,random,38.668
9	8000,ordered,0.117
0	8000,partially-ordered,17.886
1	8000,reverse-ordered,75.427

Conclusion:

Please see the Assignment3_conclusion.ipynb file for more information. Here is a preview:



Based on our understanding of the insertion sort, we know that the ordered array should take least steps and least time; the reverse-order array is the worst case; the randomly-ordered one should be in between these two, while the partially-order in between the random and the best case (cause we set it to be half ordered, half random for simplicity). This plot confirms this order of complexity for insertion sort we know across these four kinds of integer arrays.

For the reverse-ordered array, each iteration compares adjacent elements and swap. Every element must move all previous elements before inserting itself, meaning that each element swaps i times where i is its position. So the total number of swaps is $O(n^2)$ with each swap gives at least 2 hits of the array elements. The plot confirms our knowledge that it takes most time, with a curve showing the shape of a quadratic growth.

For the ordered array, each iteration only compares adjacent elements but never swaps (saving at least 2 "hits" of the array elements in swap for each iteration). As each element is compared once and break the nested loop, we have $O(n)$ comparisons with 0 swaps. The plot confirms our knowledge that it takes least time, while it's too close to the x axis, it's hard to read the details.

For the randomly-ordered array, it's something in between the first two cases, and on average we can think of each elements needs to swaps half the elements before it, while it's still $O(n^2)$ neglecting the coefficients. The plot confirms our knowledge that its curve lays in between the first two (with a slope in between the first two too), with a curve still shows some shape of a quadratic growth.

For the partially-ordered array, it's something in between the last two cases (ordered and randomly-ordered), because we set it to be half ordered and half random for simplicity. The plot confirms our knowledge that its curve lays in between the last two (with a slope in between the first two too), with a curve still shows some shape of a quadratic growth.